

Testing

Writing code you can trust

Associate Professor Jimmy J. Nielsen · Day 3 — Scientific Computing Using Python

Day 3 — Friday 13 June

- 8:30 **Testing** — pytest, parametrize, mutation testing, float comparisons ← [here](#)
- 10:00 **Verifying AI-generated code** — bisection walkthrough, 5 verification strategies
- 11:30 **Reproducible research and software development** — levels of reproducibility, 5-step AI-era guide
- 12:00 Lunch
- 12:45 **Code quality and documentation** — Sphinx, refactoring, cyclomatic complexity, pylint
- 13:45 **Collaborative version control** — branches, pull requests, Zenodo/DOI, project hand-in
- 14:30 **Students work on project — iteration 1**

Why test?

Testing is how you know whether your code does what you think it does.

This matters more, not less, when AI writes the code:

- AI-generated code **looks** correct and runs without errors — logical errors are silent
- A passing test suite is evidence that the code behaves correctly on the cases you have checked
- Tests are reusable: run them every time the code changes (including after an AI edit)

Pragmatic principle: all testing is better than no testing. But a passing test only tells you the code worked on the cases you tested — choose them carefully.

Once a bug is found, write a test to catch it. Then fix the code and confirm it passes.

What to test and when

Test levels (from small to large):

- **Unit test** — one function in isolation: given this input, do I get this output?
- **Integration test** — two or more modules working together
- **System test** — end-to-end: run the full script and check the final output

For this course: unit tests and integration tests cover most needs. System tests appear naturally when you verify the project end-to-end.

When to write tests:

- Before or alongside the code — forces you to think about edge cases
- Immediately when a bug is found — turn the bug into a failing test first
- After AI generates a function — tests are your primary verification tool

doctest

Tests embedded in docstrings

doctest — inline examples as tests

Write examples in your docstring using the `>>>` prompt. `doctest` checks that running them produces the shown output:

`factorial.py` in Moodle:

```
def factorial(n):  
    """Return n! for non-negative integer n.  
  
    >>> factorial(0)  
    1  
    >>> factorial(5)  
    120  
    """"  
    if n == 0:  
        return 1  
    return n * factorial(n - 1)
```

Run from the terminal: `python -m doctest factorial.py -v`

doctest tips

For floating-point results, exact matching fails — use `...` with `# doctest: +ELLIPSIS` or test with `numpy.allclose` in a separate pytest instead:

```
def mean(data):  
    """Return the arithmetic mean.  
  
    >>> mean([1, 2, 3])  
    2.0  
    >>> mean([1.0, 2.0]) # doctest: +ELLIPSIS  
    1.5...  
    """  
    return sum(data) / len(data)
```

When to use doctest: simple, concrete examples that also serve as documentation. Not ideal for edge cases, exceptions, or floating-point comparisons — use pytest for those.

Your turn — 10 minutes

`running_max.py` in Moodle.

Note: the version in the file has a deliberate bug introduced — do not use your Day 1 solution, or you will miss it.

Add doctests to the `running_max` function in that file:

1. Add at least two `>>>` examples to the docstring, including a case with a single element
2. Run `python -m doctest myfile.py` — do the tests pass?
3. There is a bug in the function — can you find it by writing a test that fails?

Instructor live-codes the fix and shows how the failing doctest pinpoints the bug.

pytest

The standard Python testing framework

pytest — the pattern

Write functions named `test_*` that use `assert` to check expected behaviour. pytest discovers and runs them automatically:

```
# test_stats.py
from mystats import mean, std # Import own modules and functions to test

def test_mean_simple():
    assert mean([1, 2, 3]) == 2.0

def test_mean_single_element():
    assert mean([42.0]) == 42.0

def test_mean_floats():
    import numpy as np
    assert np.isclose(mean([0.1, 0.2, 0.3]), 0.2)
```

Run from the terminal: `pytest` — discovers all `test_*.py` files and `test_*` functions recursively.

Testing edge cases and exceptions

Good tests check the normal case **and** the boundaries:

```
import pytest
from mystats import mean

def test_mean_empty_raises():
    with pytest.raises(ZeroDivisionError):
        mean([])

def test_mean_negative_values():
    assert mean([-1, 0, 1]) == 0.0

def test_mean_large_values():
    import numpy as np
    result = mean([1e15, 1e15 + 1])
    assert np.isclose(result, 1e15 + 0.5)
```

`pytest.raises` verifies that a specific exception is raised — essential for checking that your validation code works.

Using AI to generate tests

AI produces tests quickly. Use it — then verify the tests are actually checking what you think:

```
Prompt: "Write pytest test cases for the following function: <paste code>.  
Include normal cases, edge cases (empty input, zero, boundary values),  
and a test that the function raises ValueError for invalid input."
```

Checklist for AI-generated tests:

- Clear test name describing what it checks?
- Test for the normal case?
- Test for at least one edge case (empty, zero, boundary)?
- Float comparisons using `pytest.approx` or `np.isclose`, not `==`?

Float comparison tools:

- `pytest.approx(x)` — scalar or sequence; readable failure messages
- `np.isclose(a, b)` — two scalars
- `np.allclose(A, B)` — two arrays element-wise
- All default to relative tolerance `1e-5`

Weak tests pass on broken implementations — inspect them critically.

What AI-generated tests systematically miss

Gap	AI does	Fix
Single-path testing	Tests one typical input	<code>@pytest.mark.parametrize</code>
Float equality	<code>assert result == 0.333</code>	<code>pytest.approx</code> or <code>np.isclose</code>
Missing edge cases	Skips empty, zero, NaN	Add at least one degenerate case
Shallow output checks	Checks something returned	Assert against known exact values
No independent check	AI marks its own homework	Cross-check with a library or second AI

Closing these gaps is your job as the scientist — you know what "correct" means for this problem.

@pytest.mark.parametrize

AI assistants may generate single-case tests. `@pytest.mark.parametrize` runs one test function across a table of inputs:

```
import pytest
from mystats import mean

@pytest.mark.parametrize("values, expected", [
    ([1, 2, 3], 2.0),
    ([0, 0, 0], 0.0),
    ([-1, 1], 0.0),
    ([42], 42.0),
])
def test_mean(values, expected):
    assert mean(values) == pytest.approx(expected)
```

One test function → four test cases. Add a row to extend coverage — this is the systematic way to probe the cases AI didn't test.

Stacking `@pytest.mark.parametrize`

Stacking two decorators runs every combination automatically:

```
@pytest.mark.parametrize("n", [100, 200, 400])
@pytest.mark.parametrize("method", [midpoint, trapezoidal])
def test_convergence(method, n):
    err = abs(method(np.sin, 0, np.pi, n) - 2.0)
    assert err < 1.0 / n    # holds easily for  $O(1/n^2)$  methods
```

Six tests — all (method, n) combinations — without writing six functions.

Mutation testing — verifying that tests actually catch bugs

A passing test suite is not the same as a **good** test suite. **Mutation testing** checks whether your tests would catch a bug if one were introduced.

Introduce a small deliberate fault → run the test suite → confirm a test fails → revert.

```
def trapezoidal(f, a, b, n):  
    h = (b - a) / n  
    return h * (0.5*f(a) + sum(f(a + i*h) for i in range(1, n)) + 0.5*f(b))  
  
# Mutation: change 0.5 → 1.0 — does your test catch it?
```

If no test fails, the suite has a gap. **mutmut** automates this — it introduces mutations systematically so you never have to touch working code yourself.

Project requirement: document two mutation testing examples in your report — fault introduced, test that caught it, conclusion.

Five verification moves — and the pytest tool for each

In the next session we apply these to AI-generated code.

Move	pytest tool
Known ground truth	<code>pytest.approx</code> , <code>@pytest.mark.parametrize</code>
Plausibility	Plain <code>assert</code> with bounds
Edge and degenerate cases	<code>pytest.raises</code> , edge rows in <code>parametrize</code>
Mutation testing	Temporary fault → run → revert
Independent cross-check	Compare against library or a second AI

The tools are already in your hands. In the next session we use them on code we didn't write.

Your turn — 15 minutes

`integrate.py` in Moodle — contains `midpoint` .

Write a `test_integrate.py` :

1. **Known ground truth** — integrate $f(x) = 1$ over $[0, 1]$; answer is 1.0 exactly. Use `pytest.approx` .
2. **Plausibility** — result for $f(x) = x$ over $[0, 1]$ should be between 0 and 1
3. **Parametrize** — use `@pytest.mark.parametrize` to run the $f(x) = x$ over $[0, 1]$ test (answer: 0.5) for `n` in `[10, 100, 1000]`
4. **Mutation** — manually change `i + 0.5` to `i` in `midpoint` , run your tests, confirm one fails, revert

Instructor live-codes, then uses Copilot to generate additional tests and evaluates whether they are good.

Summary

doctest

- `>>>` examples in docstrings
- Run: `python -m doctest mymodule.py`

pytest

- `test_*` functions in `test_*.py` files
- `pytest.raises(ExcType)` checks exceptions
- `pytest.approx` / `np.isclose` for floats

`@pytest.mark.parametrize`

- One test function → many cases
- Stack decorators for Cartesian product

What to test

- Normal case, edge cases, invalid input
- Close the 5 AI gaps from the table

Mutation testing

- Fault → run → confirm failure → revert
- Document two examples in your report

Running tests

```
pytest          # all tests
pytest -v       # verbose
pytest -k "mean" # filter by name
```